

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-ab0e22a13c191df12.jsonl`  
- SHA-256: `9fe0a389ff5d5a088bfbdb308173d20c1097dbdab6213934c0a49c15f86e42e61`  
- Source modified: `2026-07-04T00:52:49+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `subagents`  
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`
```

Transcript

1. user (2026-07-04T00:51:00.328Z)

Code-review finder. Target: PR #62 of sports-data-collector (C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector). Diff: `cd <repo> && gh pr diff 62`. PR branch = origin/codex/a3-golden-source-videos. It rewrites tests/test_golden_source_videos.py to require 15 eval IDs, unique 32-char MD5s, and DB path reconciliation for the original 6.

ANGLE ? test adequacy / removed-behavior. Read the OLD and NEW test (via `git show origin/master:tests/test\_golden\_source\_videos.py` vs `git show origin/codex/a3-golden-source-videos:tests/test\_golden\_source\_videos.py`). Check:

1. ****Unused allowed value****: the new test allows `source\_role` in `{"owned_original", "annotation_source", "annotation_proxy", "boxhill_run_input"}` but does any entry actually use `boxhill_run_input`? If not, it's an aspirational/dead allowed value ? is that a problem (accepts a role no entry uses, so a typo'd role could slip if it matched)?`
2. ****DB reconciliation NARROWED****: the OLD test reconciled ALL manifest video_ids against the collector DB `videos` table (``WHERE video_id IN (all)``). The NEW test only reconciles the ORIGINAL_SIX against the DB. So the 9 NEW videos' local_paths are NOT checked against any DB/registry ? they're asserted only to be non-empty strings. Is that a real coverage loss (the 9 new paths could be wrong/typo'd and no test catches it)? Was the DB reconciliation load-bearing (did it verify the path actually matches a registered video)?
3. ****No file-existence check****: does any test assert the `local\_path` files actually EXIST or that bytes/md5 match a real file? Or is the manifest purely a trust-the-author record? (Given uploads are owner-gated and paths are F:/G: drives not on CI, existence can't be checked in CI ? but is there any integrity check beyond format?)
4. ****The RUN_LOG_DERIVATIVE_NOTE_VIDEO_IDS check**** asserts GX010137/GX010141 notes contain "F-drive original found" and "_1080p.mp4" ? does this lock in the honesty note correctly, or is it brittle string-matching that would break on a reworded note?

Return up to 6 candidates as JSON [{file, line, summary, failure_scenario}]. [] if none. Final message = only the JSON array.

2. assistant (2026-07-04T00:51:02.529Z)

I'll analyze the PR for test adequacy, focusing on the specific angles you mentioned. Let me start by examining the old and new test files.

3. user (2026-07-04T00:51:03.291Z)

```
64e3b97 data: record connected Boxhill source originals  
30d3379 data: reconcile golden source manifest with n15 corpus  
bfc014f fix(crawler): skip whole match when any per-set CSV fails (R1-41) (#61)  
1423e84 git-version Roora pilot golden labels  
8303fd6 record golden source video hashes (#60)  
ead8c2a preserve artifact manifests in gitignore (#59)  
01ceda9 add ruff and coverage gates (#58)  
9ce5a20 fix: pin obsreport via git+ssh instead of bare PyPI name (audit R2-3) (#57)  
adcd206 fix(db): set SQLite busy_timeout so concurrent writers wait, not error (P9) (#53)  
6f67d65 feat(import): callable import_annotations + a maturity gate (P9) (#52)
```

```
c281f82 feat(export): add source file MD5 to the eval/training manifest (P9) (#51)
a152cf5 fix(cvat): warn on rows silently skipped by load_canonical_csv (#50)
75bb0fa docs: archive CODE_HEALTH_PLAN ? project DONE (#49)
a09a000 refactor(manifests): typed pydantic models for the export manifest (A5) (#48)
6678fb0 refactor(curate): lift sys.exit out of _guard_import into CurationError (A2) (#47)
9e56056 fix(reporting): make FAQ path configurable via env var (A6) (#46)
0ddb022 docs: Tier D triage (D8) + NEXT_STEPS for parked A1/A3 (#45)
c7c4f68 refactor(config): promote validation thresholds to config (Tier C) (#43)
288b8d7 refactor(curate): concrete type hints for annotation handling (Tier C) (#44)
049cf69 refactor(db): versioned schema migrations via PRAGMA user_version (T3) (#41)
```

4. assistant (2026-07-04T00:51:05.283Z)

Now let me get the old and new test files:

5. user (2026-07-04T00:51:06.622Z)

```
44 /tmp/old_test.py
```

6. user (2026-07-04T00:51:09.182Z)

```
94 /tmp/new_test.py
```

7. user (2026-07-04T00:51:11.037Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\khealsutra-guru\sports-data-collector.

8. user (2026-07-04T00:51:11.039Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\khealsutra-guru\sports-data-collector.

9. assistant (2026-07-04T00:51:12.594Z)

Let me use PowerShell instead:

10. user (2026-07-04T00:51:13.964Z)

```
import json
import re
import sqlite3
from pathlib import Path
```

```
REPO_ROOT = Path(__file__).resolve().parents[1]
MANIFEST_PATH = REPO_ROOT / "data" / "golden_source_videos.json"
DB_PATH = REPO_ROOT / "data" / "sports_metadata.db"
EXPECTED_VIDEO_IDS = {
    "adarsh_avi_singles",
    "kushagra_singles",
    "largetest_doubles",
    "gbaaddy",
    "testlarge_short",
    "mahadevpura_singles",
}
```

```
def test_golden_source_video_manifest_matches_registry_paths() -> None:
    manifest = json.loads(MANIFEST_PATH.read_text(encoding="utf-8"))

    assert manifest["schema"] == "sports-data-collector.golden_source_videos.v1"
    assert manifest["upload_status"] == "not_uploaded_owner_gated"
```

```

entries = manifest["videos"]
assert {entry["video_id"] for entry in entries} == EXPECTED_VIDEO_IDS

with sqlite3.connect(DB_PATH) as conn:
    rows = {
        video_id: local_path
        for video_id, local_path in conn.execute(
            "SELECT video_id, local_path FROM videos WHERE video_id IN "
            f"({' '.join('?' for _ in EXPECTED_VIDEO_IDS)}")",
            tuple(EXPECTED_VIDEO_IDS),
        )
    }

assert set(rows) == EXPECTED_VIDEO_IDS
for entry in entries:
    assert entry["local_path"] == rows[entry["video_id"]]
    assert entry["local_path"].startswith(manifest["source_root"])
    assert re.fullmatch(r"[0-9a-f]{32}", entry["md5"])
    assert entry["bytes"] > 0

```

11. user (2026-07-04T00:51:17.437Z)

```

import json
import re
import sqlite3
from pathlib import Path

```

```

REPO_ROOT = Path(__file__).resolve().parents[1]
MANIFEST_PATH = REPO_ROOT / "data" / "golden_source_videos.json"
DB_PATH = REPO_ROOT / "data" / "sports_metadata.db"
EXPECTED_VIDEO_IDS = {
    "adarsh_avi_singles",
    "kushagra_singles",
    "largetest_doubles",
    "gbaaddy",
    "testlarge_short",
    "mahadevpura_singles",
    "mahadevpura_2",
    "GX010128",
    "mahadevpura_1",
    "GX030094",
    "Badminton_BXH_2",
    "GX010141",
    "GX010137",
    "Boxhill_Doubles",
    "GX020094",
}
ORIGINAL_SIX_VIDEO_IDS = {
    "adarsh_avi_singles",
    "kushagra_singles",
    "largetest_doubles",
    "gbaaddy",
    "testlarge_short",
    "mahadevpura_singles",
}
RUN_LOG_DERIVATIVE_NOTE_VIDEO_IDS = {"GX010137", "GX010141"}

```

```

def test_golden_source_video_manifest_covers_indexer_n15_registry() -> None:
    manifest = json.loads(MANIFEST_PATH.read_text(encoding="utf-8"))

    assert manifest["schema"] == "sports-data-collector.golden_source_videos.v2"
    assert manifest["upload_status"] == "not_uploaded_owner_gated"
    assert manifest["eval_manifest"] == (

```

```

    "badminton-highlight-indexer/output/human_lovo_manifest.json"
)

entries = manifest["videos"]
assert {entry["video_id"] for entry in entries} == EXPECTED_VIDEO_IDS
assert len({entry["md5"] for entry in entries}) == len(entries)

for entry in entries:
    assert re.fullmatch(r"[0-9a-f]{32}", entry["md5"])
    assert entry["source_role"] in {
        "owned_original",
        "annotation_source",
        "annotation_proxy",
        "boxhill_run_input",
    }

    assert entry["source_status"] == "present_local"
    assert entry["local_path"]
    assert entry["bytes"] > 0
    assert entry["last_write_time"]
    assert entry["md5_source"] == "Get-FileHash"

    if entry["video_id"] in RUN_LOG_DERIVATIVE_NOTE_VIDEO_IDS:
        assert "F-drive original found" in entry["notes"]
        assert "_1080p.mp4" in entry["notes"]

def test_original_six_manifest_paths_match_collector_db() -> None:
    manifest = json.loads(MANIFEST_PATH.read_text(encoding="utf-8"))
    entries = {
        entry["video_id"]: entry
        for entry in manifest["videos"]
        if entry["video_id"] in ORIGINAL_SIX_VIDEO_IDS
    }
    assert set(entries) == ORIGINAL_SIX_VIDEO_IDS

    with sqlite3.connect(DB_PATH) as conn:
        rows = {
            video_id: local_path
            for video_id, local_path in conn.execute(
                "SELECT video_id, local_path FROM videos WHERE video_id IN "
                f"({','.join('? ' for _ in ORIGINAL_SIX_VIDEO_IDS)})",
                tuple(ORIGINAL_SIX_VIDEO_IDS),
            )
        }

    assert set(rows) == ORIGINAL_SIX_VIDEO_IDS
    source_root = manifest["source_roots"]["original_six"]
    for entry in entries.values():
        assert entry["local_path"] == rows[entry["video_id"]]
        assert entry["local_path"].startswith(source_root)

```

12. assistant (2026-07-04T00:51:19.780Z)

Now let me examine the actual manifest file to check which videos use which roles:

13. user (2026-07-04T00:51:20.864Z)

```

{
  "schema": "sports-data-collector.golden_source_videos.v2",
  "generated_at_utc": "2026-07-03T19:53:33Z",
  "purpose": "Durable source-video/proxy MD5 record for the 15-video badminton golden eval corpus. The indexer joins videos by full-file MD5; uploads remain owner-gated.",
  "eval_manifest": "badminton-highlight-indexer/output/human_lovo_manifest.json",
  "upload_status": "not_uploaded_owner_gated",

```

```

"source_roots": {
  "original_six": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded
Videos\\Training\\Golden Labelled",
  "annotation_drive": "G:\\My Drive\\Sharing\\Annotation Videos\\Golden Label Videos Request -
June 15",
  "boxhill_runs": "G:\\My Drive\\Sharing\\Badminton\\Boxhill\\2026-06-17 Local no-AI (improved
windowing)"
},
"videos": [
  {
    "video_id": "adarsh_avi_singles",
    "filename": "Adarsh and Avi.MP4",
    "local_path": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded
Videos\\Training\\Golden Labelled\\Adarsh and Avi.MP4",
    "source_role": "owned_original",
    "source_status": "present_local",
    "bytes": 11445052772,
    "last_write_time": "2026-06-01T08:38:28.0000000+05:30",
    "md5": "751b6266be89dce2e9a39946caa9325c",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "kushagra_singles",
    "filename": "KushagraYashAviFirstVideo.MP4",
    "local_path": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded
Videos\\Training\\Golden Labelled\\KushagraYashAviFirstVideo.MP4",
    "source_role": "owned_original",
    "source_status": "present_local",
    "bytes": 8058214679,
    "last_write_time": "2026-05-22T08:02:16.0000000+05:30",
    "md5": "1f8ab4ba3eea45fe43a14366f825fc2f",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "largetest_doubles",
    "filename": "LargeTestVideo.MP4",
    "local_path": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded
Videos\\Training\\Golden Labelled\\LargeTestVideo.MP4",
    "source_role": "owned_original",
    "source_status": "present_local",
    "bytes": 11531882761,
    "last_write_time": "2026-05-04T17:18:56.0000000+05:30",
    "md5": "25b29aa56ba69b7e34e35d811309bba2",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "gbaaddy",
    "filename": "gBaaddy20May2026.MP4",
    "local_path": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded
Videos\\Training\\Golden Labelled\\gBaaddy20May2026.MP4",
    "source_role": "owned_original",
    "source_status": "present_local",
    "bytes": 7075098663,
    "last_write_time": "2026-05-22T08:40:40.0000000+05:30",
    "md5": "d50da42b59dae2fdeec03e47f3b353536",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "testlarge_short",
    "filename": "TestLargeVideo.MP4",
    "local_path": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded
Videos\\Training\\Golden Labelled\\TestLargeVideo.MP4",
    "source_role": "owned_original",
    "source_status": "present_local",
    "bytes": 1510427943,

```

```

    "last_write_time": "2026-05-04T20:39:14.0000000+05:30",
    "md5": "8fe6b719603dd7a7b2b6ef543139a4a3",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "mahadevpura_singles",
    "filename": "MahadevpuraSingles.MP4",
    "local_path": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded
Videos\\Training\\Golden Labelled\\MahadevpuraSingles.MP4",
    "source_role": "owned_original",
    "source_status": "present_local",
    "bytes": 196182380,
    "last_write_time": "2026-01-16T08:00:28.0000000+05:30",
    "md5": "915473794e0909c9f95a01e4511a5e98",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "mahadevpura_2",
    "filename": "mahadevpura-2_proxy.mp4",
    "local_path": "G:\\My Drive\\Sharing\\Annotation Videos\\Golden Label Videos Request -
June 15\\[Done] Video 3 - Badminton Multicourt\\mahadevpura-2_proxy.mp4",
    "source_role": "annotation_proxy",
    "source_status": "present_local",
    "bytes": 3092000570,
    "last_write_time": "2026-06-15T15:30:33.2760000+05:30",
    "md5": "d75f66340a44cac37357cce9ab988724",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "GX010128",
    "filename": "GX010128.MP4",
    "local_path": "G:\\My Drive\\Sharing\\Annotation Videos\\Golden Label Videos Request -
June 15\\[Done] Video 13 - Badminton\\GX010128.MP4",
    "source_role": "annotation_source",
    "source_status": "present_local",
    "bytes": 2342749344,
    "last_write_time": "2026-06-16T00:25:44.0000000+05:30",
    "md5": "db84f3e65318e49aea8320de4c44f7d1",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "mahadevpura_1",
    "filename": "mahadevpura-1.MP4",
    "local_path": "G:\\My Drive\\Sharing\\Annotation Videos\\Golden Label Videos Request -
June 15\\[Done] Video 14 - Badminton\\mahadevpura-1.MP4",
    "source_role": "annotation_source",
    "source_status": "present_local",
    "bytes": 548827049,
    "last_write_time": "2026-06-16T00:22:14.0000000+05:30",
    "md5": "38bef75d78cbbd8209a80fd73c28e6ed",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "GX030094",
    "filename": "GX030094_proxy.mp4",
    "local_path": "G:\\My Drive\\Sharing\\Annotation Videos\\Golden Label Videos Request -
June 15\\[Done] Video 6 - Badminton\\GX030094_proxy.mp4",
    "source_role": "annotation_proxy",
    "source_status": "present_local",
    "bytes": 1429944539,
    "last_write_time": "2026-06-15T17:13:40.5290000+05:30",
    "md5": "27f1c707bbfb9b81d3b7c6b5afdda7e8",
    "md5_source": "Get-FileHash"
  },
  {

```

```

    "video_id": "Badminton_BXH_2",
    "filename": "Badminton BXH 2_proxy.mp4",
    "local_path": "G:\\My Drive\\Sharing\\Annotation Videos\\Golden Label Videos Request -
June 15\\[Done] Video 10 - Badminton Multicourt\\Badminton BXH 2_proxy.mp4",
    "source_role": "annotation_proxy",
    "source_status": "present_local",
    "bytes": 3660841332,
    "last_write_time": "2026-06-15T16:00:42.7790000+05:30",
    "md5": "ebebccb702c4be5f3a7da7e000b0f3a4",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "GX010141",
    "filename": "GX010141.MP4",
    "local_path": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded Videos\\Badminton -
Boxhill - 17 June\\GX010141.MP4",
    "source_role": "owned_original",
    "source_status": "present_local",
    "bytes": 5772876373,
    "last_write_time": "2026-06-17T09:11:06.0000000+05:30",
    "md5": "219a84a4e1a985edf5fb039142c18b8d",
    "md5_source": "Get-FileHash",
    "notes": "F-drive original found after the drive was connected on 2026-07-04. Earlier
Boxhill run log records derivative GX010141_1080p.mp4 with MD5 daaf8af198640516d14b217f50445d99;
this entry records the present original source file."
  },
  {
    "video_id": "GX010137",
    "filename": "GX010137.MP4",
    "local_path": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded Videos\\Badminton -
Boxhill - 17 June\\GX010137.MP4",
    "source_role": "owned_original",
    "source_status": "present_local",
    "bytes": 6762153261,
    "last_write_time": "2026-06-17T08:21:58.0000000+05:30",
    "md5": "3d19ef835a69be93acdd75ef636f345a",
    "md5_source": "Get-FileHash",
    "notes": "F-drive original found after the drive was connected on 2026-07-04. Earlier
Boxhill run log records derivative GX010137_1080p.mp4 with MD5 74eedbc35d739c231344b8929df4b4a7;
this entry records the present original source file."
  },
  {
    "video_id": "Boxhill_Doubles",
    "filename": "Boxhill Doubles_proxy.mp4",
    "local_path": "G:\\My Drive\\Sharing\\Annotation Videos\\Golden Label Videos Request -
June 15\\[Done] Video 12 - Badminton Multicourt\\Boxhill Doubles_proxy.mp4",
    "source_role": "annotation_proxy",
    "source_status": "present_local",
    "bytes": 1553998715,
    "last_write_time": "2026-06-15T16:23:18.9720000+05:30",
    "md5": "9a35fffdfbdee397575ffda9d68ebca3",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "GX020094",
    "filename": "GX020094_proxy.mp4",
    "local_path": "G:\\My Drive\\Sharing\\Annotation Videos\\Golden Label Videos Request -
June 15\\[Done] Video 4 - Badminton\\GX020094_proxy.mp4",
    "source_role": "annotation_proxy",
    "source_status": "present_local",
    "bytes": 1554122102,
    "last_write_time": "2026-06-15T16:54:20.8510000+05:30",
    "md5": "08471270f285123021ef4f9d21d1b05b",
    "md5_source": "Get-FileHash"
  }
}

```

```
]
}
```

14. assistant (2026-07-04T00:51:22.948Z)

Perfect! Now let me check if there are any references to the allowed roles in code to understand if `boxhill\_run\_input` is actually used:

15. user (2026-07-04T00:51:53.659Z)

```
./tests/test_golden_source_videos.py:          "boxhill_run_input",
Binary file ./tests/__pycache__/test_golden_source_videos.cpython-313-pytest-9.0.3.pyc matches
```

16. user (2026-07-04T00:51:56.346Z)

```
data/golden_source_videos.json:      "source_role": "owned_original",
data/golden_source_videos.json:      "source_role": "owned_original",
data/golden_source_videos.json:      "source_role": "owned_original",
data/golden_source_videos.json:      "source_role": "owned_original",
data/golden_source_videos.json:      "source_role": "owned_original",
data/golden_source_videos.json:      "source_role": "owned_original",
data/golden_source_videos.json:      "source_role": "annotation_proxy",
data/golden_source_videos.json:      "source_role": "annotation_source",
data/golden_source_videos.json:      "source_role": "annotation_source",
data/golden_source_videos.json:      "source_role": "annotation_proxy",
data/golden_source_videos.json:      "source_role": "annotation_proxy",
data/golden_source_videos.json:      "source_role": "owned_original",
data/golden_source_videos.json:      "source_role": "owned_original",
data/golden_source_videos.json:      "source_role": "annotation_proxy",
data/golden_source_videos.json:      "source_role": "annotation_proxy",
```

17. assistant (2026-07-04T00:51:58.530Z)

Now let me check the old test's reconciliation behavior more carefully and understand what was checked:

18. user (2026-07-04T00:51:59.592Z)

```
30d3379 data: reconcile golden source manifest with n15 corpus
1423e84 git-version Roora pilot golden labels
8303fd6 record golden source video hashes (#60)
6f67d65 feat(import): callable import_annotations + a maturity gate (P9) (#52)
ca61405 docs(annotation): ROORA_RUNBOOK - the operator e2e (send -> receive -> golden labels)
(#25)
a068fd1 feat(curate): prep-annotation-proxy ? frame-identical vendor handoff re-encode (ADR-002)
(#22)
9e6f204 feat(corpus): ingest the n=6 owned golden corpus (262 rallies, Proprietary/commercial)
(#19)
4390a9d fix(import-local): protect golden labels + guard the fractional-rally_number landmine
(#15)
099cf67 Manual-ingestion: normalize-annotations guard + positioning doc (#13)
0e4ccc7 Merge pull request #12 from avidullu/feat/collector-report-refinements
```

19. assistant (2026-07-04T00:52:01.193Z)

Let me check the manifest structure and what the old vs new tests expect:

20. user (2026-07-04T00:52:02.245Z)

```
{
  "schema": "sports-data-collector.golden_source_videos.v2",
  "generated_at_utc": "2026-07-03T19:53:33Z",
  "purpose": "Durable source-video/proxy MD5 record for the 15-video badminton golden eval
corpus. The indexer joins videos by full-file MD5; uploads remain owner-gated.",
```

```

"eval_manifest": "badminton-highlight-indexer/output/human_lovo_manifest.json",
"upload_status": "not_uploaded_owner_gated",
"source_roots": {
  "original_six": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded
Videos\\Training\\Golden Labelled",
  "annotation_drive": "G:\\My Drive\\Sharing\\Annotation Videos\\Golden Label Videos Request -
June 15",
  "boxhill_runs": "G:\\My Drive\\Sharing\\Badminton\\Boxhill\\2026-06-17 Local no-AI (improved
windowing)"
},
"videos": [
  {
    "video_id": "adarsh_avi_singles",
    "filename": "Adarsh and Avi.MP4",
    "local_path": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded
Videos\\Training\\Golden Labelled\\Adarsh and Avi.MP4",
    "source_role": "owned_original",
    "source_status": "present_local",
    "bytes": 11445052772,
    "last_write_time": "2026-06-01T08:38:28.0000000+05:30",
    "md5": "751b6266be89dce2e9a39946caa9325c",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "kushagra_singles",
    "filename": "KushagraYashAviFirstVideo.MP4",
    "local_path": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded
Videos\\Training\\Golden Labelled\\KushagraYashAviFirstVideo.MP4",
    "source_role": "owned_original",
    "source_status": "present_local",
    "bytes": 8058214679,
    "last_write_time": "2026-05-22T08:02:16.0000000+05:30",
    "md5": "1f8ab4ba3eea45fe43a14366f825fc2f",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "largetest_doubles",
    "filename": "LargeTestVideo.MP4",
    "local_path": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded
Videos\\Training\\Golden Labelled\\LargeTestVideo.MP4",
    "source_role": "owned_original",
    "source_status": "present_local",
    "bytes": 11531882761,
    "last_write_time": "2026-05-04T17:18:56.0000000+05:30",
    "md5": "25b29aa56ba69b7e34e35d811309bba2",
    "md5_source": "Get-FileHash"
  },
  {
    "video_id": "gbaaddy",
    "filename": "gBaaddy20May2026.MP4",
    "local_path": "F:\\[Khelsutra] GoPro Hero Black 12\\GoPro Recorded
Videos\\Training\\Golden Labelled\\gBaaddy20May2026.MP4",
    "source_role": "owned_original",

```

21. assistant (2026-07-04T00:52:04.743Z)

Now let me analyze the key differences between the old test and new test more carefully:

22. user (2026-07-04T00:52:05.732Z)

```

diff --git a/tests/test_golden_source_videos.py b/tests/test_golden_source_videos.py
index 8e757e9..cc6fcdd 100644
--- a/tests/test_golden_source_videos.py
+++ b/tests/test_golden_source_videos.py
@@ -14,31 +14,81 @@ EXPECTED_VIDEO_IDS = {

```

```

        "gbaaddy",
        "testlarge_short",
        "mahadevpura_singles",
+       "mahadevpura_2",
+       "GX010128",
+       "mahadevpura_1",
+       "GX030094",
+       "Badminton_BXH_2",
+       "GX010141",
+       "GX010137",
+       "Boxhill_Doubles",
+       "GX020094",
    }
+ORIGINAL_SIX_VIDEO_IDS = {
+    "adarsh_avi_singles",
+    "kushagra_singles",
+    "largetest_doubles",
+    "gbaaddy",
+    "testlarge_short",
+    "mahadevpura_singles",
+}
+RUN_LOG_DERIVATIVE_NOTE_VIDEO_IDS = {"GX010137", "GX010141"}

-def test_golden_source_video_manifest_matches_registry_paths() -> None:
+def test_golden_source_video_manifest_covers_indexer_n15_registry() -> None:
    manifest = json.loads(MANIFEST_PATH.read_text(encoding="utf-8"))

-    assert manifest["schema"] == "sports-data-collector.golden_source_videos.v1"
+    assert manifest["schema"] == "sports-data-collector.golden_source_videos.v2"
    assert manifest["upload_status"] == "not_uploaded_owner_gated"
+    assert manifest["eval_manifest"] == (
+        "badminton-highlight-indexer/output/human_lovo_manifest.json"
+    )

    entries = manifest["videos"]
    assert {entry["video_id"] for entry in entries} == EXPECTED_VIDEO_IDS
+    assert len({entry["md5"] for entry in entries}) == len(entries)
+
+    for entry in entries:
+        assert re.fullmatch(r"[0-9a-f]{32}", entry["md5"])
+        assert entry["source_role"] in {
+            "owned_original",
+            "annotation_source",
+            "annotation_proxy",
+            "boxhill_run_input",
+        }
+
+        assert entry["source_status"] == "present_local"
+        assert entry["local_path"]
+        assert entry["bytes"] > 0
+        assert entry["last_write_time"]
+        assert entry["md5_source"] == "Get-FileHash"
+
+        if entry["video_id"] in RUN_LOG_DERIVATIVE_NOTE_VIDEO_IDS:
+            assert "F-drive original found" in entry["notes"]
+            assert "_1080p.mp4" in entry["notes"]
+
+def test_original_six_manifest_paths_match_collector_db() -> None:
+    manifest = json.loads(MANIFEST_PATH.read_text(encoding="utf-8"))
+    entries = {
+        entry["video_id"]: entry
+        for entry in manifest["videos"]
+        if entry["video_id"] in ORIGINAL_SIX_VIDEO_IDS
    }

```

```

+     }
+     assert set(entries) == ORIGINAL_SIX_VIDEO_IDS

    with sqlite3.connect(DB_PATH) as conn:
        rows = {
            video_id: local_path
            for video_id, local_path in conn.execute(
                "SELECT video_id, local_path FROM videos WHERE video_id IN "
-                f"({''.join('?' for _ in EXPECTED_VIDEO_IDS)}))",
-                tuple(EXPECTED_VIDEO_IDS),
+                f"({''.join('?' for _ in ORIGINAL_SIX_VIDEO_IDS)}))",
+                tuple(ORIGINAL_SIX_VIDEO_IDS),
            )
        }

-     assert set(rows) == EXPECTED_VIDEO_IDS
-     for entry in entries:
+     assert set(rows) == ORIGINAL_SIX_VIDEO_IDS
+     source_root = manifest["source_roots"]["original_six"]
+     for entry in entries.values():
+         assert entry["local_path"] == rows[entry["video_id"]]
-         assert entry["local_path"].startswith(manifest["source_root"])
-         assert re.fullmatch(r"[0-9a-f]{32}", entry["md5"])
-         assert entry["bytes"] > 0
+         assert entry["local_path"].startswith(source_root)

```

23. assistant (2026-07-04T00:52:07.627Z)

Excellent! Now let me count the actual source_role usage:

24. user (2026-07-04T00:52:10.126Z)

Source roles used in manifest:

```

annotation_proxy: 5 videos
['mahadevpura_2', 'GX030094', 'Badminton_BXH_2', 'Boxhill_Doubles', 'GX020094']
annotation_source: 2 videos
['GX010128', 'mahadevpura_1']
owned_original: 8 videos
['adarsh_avi_singles', 'kushagra_singles', 'largetest_doubles', 'gbaaddy',
'testlarge_short', 'mahadevpura_singles', 'GX010141', 'GX010137']

```

Allowed but unused roles:

```
boxhill_run_input
```

25. assistant (2026-07-04T00:52:12.184Z)

Perfect! Now let me check the notes field more carefully for the two videos:

26. user (2026-07-04T00:52:13.921Z)

GX010141:

notes: F-drive original found after the drive was connected on 2026-07-04. Earlier Boxhill run log records derivative GX010141_1080p.mp4 with MD5 daaf8af198640516d14b217f50445d99; this entry records the present original source file.

GX010137:

notes: F-drive original found after the drive was connected on 2026-07-04. Earlier Boxhill run log records derivative GX010137_1080p.mp4 with MD5 74eedbc35d739c231344b8929df4b4a7; this entry records the present original source file.

27. assistant (2026-07-04T00:52:15.759Z)

Now let me check if the notes field is optional in the new test:

28. user (2026-07-04T00:52:17.110Z)

```
if entry["video_id"] in RUN_LOG_DERIVATIVE_NOTE_VIDEO_IDS:
    assert "F-drive original found" in entry["notes"]
    assert "_1080p.mp4" in entry["notes"]
```

```
def test_original_six_manifest_paths_match_collector_db() -> None:
    manifest = json.loads(MANIFEST_PATH.read_text(encoding="utf-8"))
    entries = {
        entry["video_id"]: entry
        for entry in manifest["videos"]
        if entry["video_id"] in ORIGINAL_SIX_VIDEO_IDS
```

29. assistant (2026-07-04T00:52:19.360Z)

Now let me check what the test expects when accessing `entry["notes"]` without checking if the field exists first:

30. user (2026-07-04T00:52:20.870Z)

Test would FAIL with KeyError: 'notes'

31. assistant (2026-07-04T00:52:23.000Z)

Good. Now let me check if there are any issues with the 9 new videos not being checked against the DB. Let me see what the purpose is:

32. user (2026-07-04T00:52:25.867Z)

```
Original 6: 6
All 15: 15
New 9 not reconciled against DB:
  Badminton_BXH_2: role=annotation_proxy, path=G:\My Drive\Sharing\Annotation Videos\Golden
Label Videos Re...
  Boxhill_Doubles: role=annotation_proxy, path=G:\My Drive\Sharing\Annotation Videos\Golden
Label Videos Re...
  GX010128: role=annotation_source, path=G:\My Drive\Sharing\Annotation Videos\Golden Label
Videos Re...
  GX010137: role=owned_original, path=F:\[Khelsutra] GoPro Hero Black 12\GoPro Recorded
Videos\Bad...
  GX010141: role=owned_original, path=F:\[Khelsutra] GoPro Hero Black 12\GoPro Recorded
Videos\Bad...
  GX020094: role=annotation_proxy, path=G:\My Drive\Sharing\Annotation Videos\Golden Label
Videos Re...
  GX030094: role=annotation_proxy, path=G:\My Drive\Sharing\Annotation Videos\Golden Label
Videos Re...
  mahadevpura_1: role=annotation_source, path=G:\My Drive\Sharing\Annotation Videos\Golden Label
Videos Re...
  mahadevpura_2: role=annotation_proxy, path=G:\My Drive\Sharing\Annotation Videos\Golden Label
Videos Re...
```

33. assistant (2026-07-04T00:52:27.714Z)

Now let me check if the old test's reconciliation would have caught issues with the 9 new videos:

34. user (2026-07-04T00:52:28.623Z)

```
Exit code 127
/usr/bin/bash: line 1: sqlite3: command not found
```

35. user (2026-07-04T00:52:34.794Z)

Total videos in DB: 31

Video IDs in DB:

adarsh_avi_singles
gbaaddy
kushagra_singles
largetest_doubles
mahadevpura_singles
my_custom_local_match
shuttleset_1
shuttleset_10
shuttleset_11
shuttleset_12
shuttleset_13
shuttleset_14
shuttleset_15
shuttleset_16
shuttleset_17
shuttleset_18
shuttleset_19
shuttleset_2
shuttleset_20
shuttleset_21
shuttleset_22
shuttleset_23
shuttleset_24
shuttleset_25
shuttleset_3
shuttleset_4
shuttleset_6
shuttleset_8
shuttleset_9
staging_match_non_comm
testlarge_short

36. assistant (2026-07-04T00:52:48.888Z)

Perfect! So the DB only contains the 6 original videos (plus other videos), not the 9 new ones. Now I have all the information I need. Let me compile the findings:

```
```json
[
 {
 "file": "tests/test_golden_source_videos.py",
 "line": 45,
 "summary": "Unused allowed source_role value 'boxhill_run_input': test allows this role, but NO video in the manifest uses it. A typo'd role like 'boxhill_run_inpu' would fail validation, but 'boxhill_run_input' would silently pass even if no entry actually uses it.",
 "failure_scenario": "Future manifest entry with typo in source_role (e.g. 'boxhill_run_input_v2' or similar) could slip through if it happens to match one of the allowed values due to the role being aspirational rather than enforced per-entry."
 },
 {
 "file": "tests/test_golden_source_videos.py",
 "line": 70,
 "summary": "DB reconciliation NARROWED: old test reconciled ALL 6 manifest videos against DB; new test reconciles only the ORIGINAL_SIX (6 videos) against DB. The 9 NEW videos (GX010128, GX010137, GX010141, GX030094, Badminton_BXH_2, Boxhill_Doubles, GX020094, mahadevpura_1, mahadevpura_2) are NOT checked against the DB. Their local_paths are only asserted to be non-empty strings, not validated against any registry.",
 "failure_scenario": "A typo'd local_path in one of the 9 new videos (e.g., 'G:\\\\My Drive\\\\Sharing\\\\Annotation Videos\\\\Golden Label Videos Request - June 15\\\\[Done] Video 13 - Badminton\\\\GX010128_TYP0.MP4') would pass the test because only format validation (non-empty string) is done, not DB reconciliation."
 }
],
```

```

{
 "file": "tests/test_golden_source_videos.py",
 "line": 51,
 "summary": "Missing file existence check: test asserts local_path is a non-empty string and md5 matches regex format, but does NOT check that the file actually exists on disk. Manifest is purely trust-the-author (understandable on CI which lacks F:/G: drive access), but no integrity check beyond format.",
 "failure_scenario": "An entry with a fabricated local_path like 'F:\\\\[Khelsutra] GoPro Hero Black 12\\\\nonexistent_file.MP4' and a fake MD5 like '0000000000000000000000000000aaaa' would pass all format checks. The manifest relies entirely on author honesty."
},
{
 "file": "tests/test_golden_source_videos.py",
 "line": 54,
 "summary": "RUN_LOG_DERIVATIVE_NOTE check uses brittle substring matching: test asserts that entry[\"notes\"] contains 'F-drive original found' AND '_1080p.mp4', but does not validate the semantic structure or completeness of the notes. A reworded explanation would break the test.",
 "failure_scenario": "If the notes message is rephrased to 'Original file from F-drive found after connection' and '_1080p.mp4_version.mp4', the substrings 'F-drive original found' and '_1080p.mp4' would no longer be found, causing the test to fail despite the notes being semantically equivalent and accurate."
},
{
 "file": "tests/test_golden_source_videos.py",
 "line": 54,
 "summary": "Missing optional field check: test accesses entry[\"notes\"] without checking if the field exists, assuming it is always present for the RUN_LOG_DERIVATIVE_NOTE_VIDEO_IDS videos. If a future entry is added to that set but lacks the notes field, a KeyError will be raised.",
 "failure_scenario": "Adding 'GX010128' to RUN_LOG_DERIVATIVE_NOTE_VIDEO_IDS but not ensuring its manifest entry has a 'notes' field would cause the test to crash with KeyError: 'notes'."
},
{
 "file": "tests/test_golden_source_videos.py",
 "line": 27,
 "summary": "MD5 uniqueness check is shallow: test asserts len({entry[\"md5\"] for entry in entries}) == len(entries), confirming all 15 MD5s are unique. However, it does NOT validate that each MD5 actually corresponds to the file at local_path (would require file I/O, infeasible on CI). So duplicate MD5s would be caught, but wrong MD5s would not.",
 "failure_scenario": "If two entries swap their MD5 values (e.g., GX010137 takes GX010141's MD5 and vice versa), all uniqueness and format checks pass, but the indexer's join-by-MD5 logic would silently produce incorrect results when matching videos across sources."
}
]
...

```